

[6] Enhancing RAG-Based MCQ Generation for Java Programming Education: A Modular Evaluation of Chunking, Retrieval and LLM Performance

- **Authors:** E. Olibo
 - **Venue & Year:** Bachelor's Thesis, Kristianstad University (2025)
 - **Category:** Programming-Specific RAG
 - **Associated Full Paper PDF:** Reference [6] (Kristianstad DiVA)
-

1-Minute Executive Takeaway

Key Finding: Confirms code-specialized LLMs outperform general LLMs for programming MCQs, justifying our selection of Qwen2.5-Coder.

Core Research Objective

To conduct an extensive modular evaluation comparing various text chunking strategies, vector retrieval top-K configurations, and instruction-tuned LLMs for generating Java programming MCQs.

Methodology & Architecture

Benchmarked multiple embedding models and open-weight LLMs across diverse chunking sizes (256, 512, 1024 tokens) and evaluated question quality across syntax, distractors, and difficulty.

Key Empirical Findings & Evidence

Demonstrated that programming MCQ generation requires specialized instruction-tuned code models (such as Qwen-Coder or CodeLlama) and that bounded context windows prevent retrieval noise from corrupting code syntax.

Direct Relevance to Our Kwiz Paper

Cited in Section 2.2 as closely related domain work. Because Olibo thoroughly explored hyperparameter sweeps for Java, our study fixes a single optimal deployment configuration (`Qwen2.5-Coder-7B-Instruct` + `nomic-embed-text`) and investigates operational systems metrics: latency, caching, and concurrency.